

Promise Chaining

Ek ke baad ek async kaam karne ho to promises me **chaining** hoti hai.

Chain kaise chalti hai

Har `.then` ek **naya promise** return karta hai. Handler jo value return karta hai wo agle `.then` ko mil jaati hai.

javascript

```
new Promise(resolve => setTimeout(() => resolve(1), 1000))
  .then(result => { console.log(result); return result * 2; }) // 1
  .then(result => { console.log(result); return result * 2; }) // 2
  .then(result => { console.log(result); return result * 2; }); // 4
```

Galti: yeh chaining nahi hai

Ek hi promise par kai `.then` lagana chain nahi hai. Sab **alag alag, same result** dekhte hain (ek doosre ko value pass nahi karte).

javascript

```
promise.then(f1);
promise.then(f2); // dono ko promise ka wahi result milta hai
```

Handler me promise return karna

Handler promise return kare to agla `.then` uske **settle hone tak wait** karta hai, phir uska result leta hai. Isse async kaam ek ke baad ek hote hain.

javascript

```
.then(result => new Promise(resolve => setTimeout(() => resolve(result * 2), 10
```

Example: scripts ek ke baad ek

javascript

```
loadScript("one.js")
  .then(() => loadScript("two.js"))
  .then(() => loadScript("three.js"))
  .then(() => { one(); two(); three(); });
```

Code **neeche** badhta hai, daayin taraf nahi. Pyramid of doom nahi banta.

Nested `.then` (andar `.then`) likhna bhi possible hai, par wo phir se daayin taraf badhta hai. Aam taur par chaining behtar hai.

Thenables

Agar handler koi bhi aisa object return kare jisme `.then` method ho, to JS use promise jaisa hi treat karta hai. Ye third-party libraries ke promise-compatible objects ke liye hai.

Bada example: `fetch`

javascript

```
fetch("user.json")
  .then(response => response.json()) // JSON parse
  .then(user => fetch(`https://api.github.com/users/${user.name}`))
  .then(response => response.json())
  .then(githubUser => { /* avatar dikhao */ });
```

- `fetch(url)` promise deta hai jo headers aate hi resolve hota hai.
- `response.json()` / `response.text()` bhi promise dete hain (poora data aane par resolve).

Chain ko aage badhane layak rakho

Agar avatar dikhane ke baad kuch aur karna ho, to avatar wala step ek **promise return** kare jo tab resolve ho jab kaam poora ho:

javascript

```
.then(githubUser => new Promise(resolve => {
  // avatar dikhao
  setTimeout(() => { /* hatao */ resolve(githubUser); }, 3000);
}))
.then(githubUser => console.log(`Done: ${githubUser.name}`));
```

Achhi aadat: async kaam hamesha promise return kare, taaki baad me chain badha sako.

Code ko chhote reusable functions me todo:

javascript

```
function loadJson(url) { return fetch(url).then(r => r.json()); }  
function loadGithubUser(name) { return loadJson(`https://api.github.com/users/$  
  
loadJson("user.json")  
  .then(user => loadGithubUser(user.name))  
  .then(showAvatar);
```

Task: `then(f1).catch(f2)` VS `then(f1, f2)`

Ye **barabar nahi** hain.

- `.then(f1).catch(f2)`: `f1` me error aaye to `catch` pakadta hai.
- `.then(f1, f2)`: `f2` sirf pichhle promise ke reject ko sambhalta hai, `f1` ke error ko nahi (neeche koi handler nahi).

Yaad rakho

- Har `.then` naya promise deta hai, return value aage jaati hai.
- Promise return karo to chain us par wait karti hai.
- Chain flat rakho, nesting nahi.